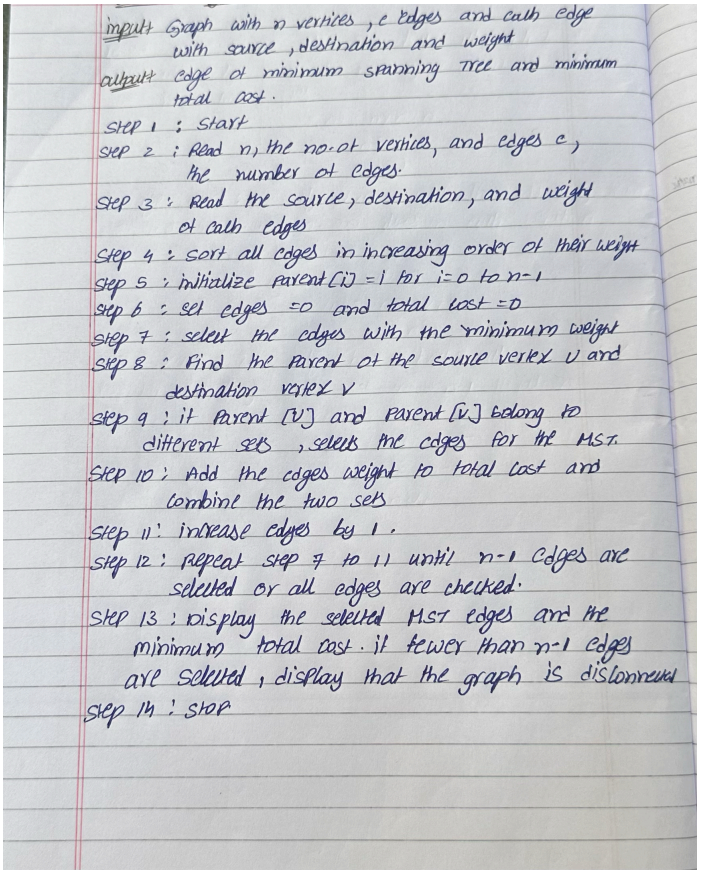
PRACTICAL - 10

AIM : Implement Kruskal's algorithm.

ALGORITHM :



input: Graph with n vertices, e edges and each edge with source, destination and weight
output: edge of minimum spanning tree and minimum total cost.

Step 1 : Start
Step 2 : Read n, the no. of vertices, and edges e, the number of edges.
Step 3 : Read the source, destination, and weight of each edges
Step 4 : sort all edges in increasing order of their weight
Step 5 : initialize parent[i] = i for i=0 to n-1
Step 6 : set edges =0 and total cost =0
Step 7 : select the edges with the minimum weight
Step 8 : Find the parent of the source vertex U and destination vertex V
Step 9 : if Parent [U] and Parent [V] belong to different sets, selects the edges for the MST.
Step 10 : Add the edges weight to total cost and combine the two sets
Step 11 : increase edges by 1.
Step 12 : Repeat step 7 to 11 until n-1 edges are selected or all edges are checked.
Step 13 : Display the selected MST edges and the minimum total cost. if fewer than n-1 edges are selected, display that the graph is disconnected
Step 14 : Stop

PROGRAM :

```
#include <iostream>
#include <algorithm>
using namespace std;

class Graph
{
public:
    int cost[10][10];
    int n;

    void getInput()
```

```
{
    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter cost adjacency matrix:\n";

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> cost[i][j];
        }
    }
}

int findParent(int parent[], int vertex)
{
    if (parent[vertex] == vertex)
        return vertex;

    return parent[vertex] = findParent(parent, parent[vertex]);
}

void kruskal()
{
    int parent[10];
    int edges = 0;
    int totalCost = 0;

    for (int i = 0; i < n; i++)
    {
        parent[i] = i;
    }

    // Store edges as {weight, source, destination}
    int edgeList[45][3];
    int edgeCount = 0;
```

```
for (int i = 0; i < n; i++)

 {
        for (int j = i + 1; j < n; j++)
        {
           if (cost[i][j] != 0)
           {
              edgeList[edgeCount][0] = cost[i][j];
              edgeList[edgeCount][1] = i;
              edgeList[edgeCount][2] = j;
              edgeCount++;
           }
        }
     }

     // Sort edges by weight using bubble sort
     for (int i = 0; i < edgeCount - 1; i++)
     {
        for (int j = 0; j < edgeCount - i - 1; j++)
        {
           if (edgeList[j][0] > edgeList[j + 1][0])
           {
              for (int k = 0; k < 3; k++)
              {
                 swap(edgeList[j][k], edgeList[j + 1][k]);
              }
           }
        }
     }

     cout << "\nEdges in Minimum Spanning Tree:\n";

     for (int i = 0; i < edgeCount && edges < n - 1; i++)
     {
        int weight = edgeList[i][0];
        int u = edgeList[i][1];
        int v = edgeList[i][2];
```

```
    int parentU = findParent(parent, u);
        int parentV = findParent(parent, v);


  if (parentU != parentV)
        {
            cout << u << " - " << v << " = " << weight << endl;

            parent[parentU] = parentV;
            totalCost += weight;
            edges++;
        }
    }

    if (edges != n - 1)
    {
        cout << "Graph is disconnected. MST cannot be formed.\n";
        return;
    }

    cout << "\nMinimum Cost = " << totalCost << endl;
  }
};

int main()
{
  Graph g;

  g.getInput();
  g.kruskal();

  return 0;
}
```

OUTPUT:

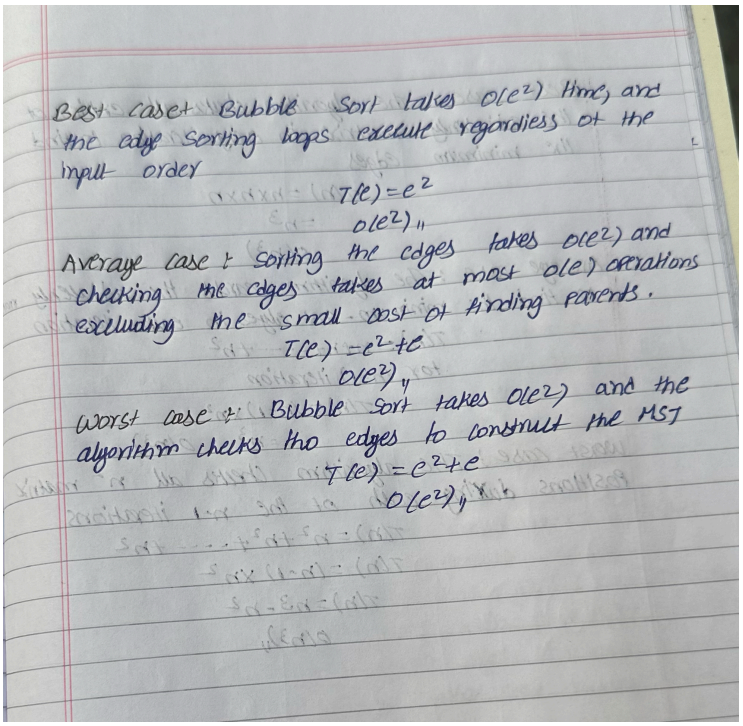
```
Output

Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0

Edges in Minimum Spanning Tree:
2 - 3 = 4
0 - 3 = 5
0 - 1 = 10

Minimum Cost = 19
```

TIME COMPLEXITY :



CONCLUSION :

I successfully implemented Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of a weighted, undirected graph. I learned how to sort edges in ascending order of their weights and select the minimum-cost edges without forming cycles. Through this practical, I understood the working of a greedy algorithm and the use of the Union-Find technique to detect cycles. I also learned how to calculate the minimum total cost required to connect all vertices in a graph.